

ATAQUE DOS

Adrià Trillo y Nicolás Zabala

ENTI-UB REDES III

1 CONTENIDO

2	Introducción.....	2
3	Entorno de trabajo y planteamiento del laboratorio.....	3
4	¿Qué es un ataque DoS y cuáles son sus diferentes variantes?.....	3
4.1	DoS volumétrico.....	4
4.2	DoS a nivel de protocolo.....	4
4.3	DoS a nivel de aplicación.....	5
4.4	DDoS	5
4.5	Relación con nuestra práctica	6
5	Desarrollo técnico de lo que hemos hecho en el laboratorio	6
5.1	Preparación de la máquina víctima.....	6
5.2	Preparación de la máquina atacante	7
5.3	Lo que hemos observado en las capturas y resultados.....	8
6	Contramedidas básicas.....	8
6.1	SYN cookies.....	9
6.2	Limitación de conexiones y rate limiting	9
6.3	Firewalls e IDS/IPS.....	9
6.4	Ajuste de parámetros del kernel TCP.....	10
6.5	Balanceadores y protección perimetral.....	10
6.6	Monitorización continua	10
7	PARTE PRÁCTICA.....	11
7.1	Apache	12
7.2	STATS NORMALES	13
7.3	ab	14
7.4	DoS.....	15
7.5	Wireshark.....	17
8	Incidencias técnicas y soluciones.....	17
8.1	Limitación del entorno Ubuntu Server	18
8.1.1	¿Qué hemos hecho?.....	18
8.1.2	Solución aplicada.....	18
8.2	Tamaño excesivo del archivo .pcap.....	18
8.2.1	¿Qué hemos observado?.....	18
8.2.2	¿Qué hemos hecho para resolverlo?	19
9	Conclusión.....	19

2 INTRODUCCIÓN

En esta práctica hemos trabajado sobre uno de los ataques clásicos contra la disponibilidad de un servicio: el ataque de denegación de servicio, también conocido como DoS. Lo que hemos hecho aquí ha sido montar un pequeño laboratorio controlado con dos máquinas virtuales, una haciendo de atacante y otra haciendo de víctima, para comprobar de forma práctica cómo un servidor web puede verse afectado cuando recibe una gran cantidad de solicitudes de conexión maliciosas.

Nuestro objetivo no ha sido “tirar” un servicio real ni probar nada fuera del entorno de laboratorio, sino entender qué ocurre técnicamente, qué efecto tiene el ataque sobre el sistema, cómo se observa ese efecto en las herramientas de monitorización y qué medidas básicas pueden aplicarse para reducir el impacto.

En concreto, hemos trabajado con un ataque SYN Flood, que pertenece a la familia de los ataques DoS a nivel de protocolo. Durante la práctica hemos preparado la máquina víctima con Apache, hemos dejado corriendo procesos de monitorización en segundo plano para registrar CPU, conexiones y latencia, y desde la máquina atacante hemos lanzado un flujo masivo de paquetes SYN utilizando hping3. Después hemos comprobado el impacto tanto a nivel de servicio como a nivel de red, y hemos identificado varias incidencias técnicas reales durante la realización del laboratorio.

3 ENTORNO DE TRABAJO Y PLANTEAMIENTO DEL LABORATORIO

Para realizar la práctica hemos utilizado dos máquinas virtuales dentro de un entorno aislado. La idea ha sido separar claramente los papeles de cada sistema:

- **Máquina atacante:** Parrot/Kali Linux.
- **Máquina víctima:** Ubuntu Server con Apache instalado y funcionando.
- **Servicio atacado:** servidor web escuchando en el puerto 80.

Herramientas empleadas: hping3, ab (ApacheBench), curl, tcpdump, sar, ss y htop.

Lo que hemos hecho en la máquina víctima ha sido dejar preparado un servidor web real y un sistema de registro continuo. Como Ubuntu Server no tenía entorno gráfico cómodo para trabajar con varias ventanas al mismo tiempo, en vez de abrir múltiples terminales hemos optado por una solución mucho más adecuada para este escenario: redirigir la salida a archivos y ejecutar los procesos en segundo plano. Esa decisión nos ha permitido registrar toda la actividad sin depender de varias sesiones abiertas, y además nos ha servido para generar evidencias directamente reutilizables en el informe.

En la máquina atacante hemos preparado las herramientas necesarias para dos tareas distintas. Por un lado, generar el ataque SYN Flood con hping3. Por otro, lanzar peticiones legítimas con ab y curl para comprobar cómo se comportaba el servidor mientras estaba bajo presión.

4 ¿QUÉ ES UN ATAQUE DoS Y CUÁLES SON SUS DIFERENTES VARIANTES?

Un ataque DoS (Denial of Service) o ataque de denegación de servicio es un ataque cuyo objetivo es conseguir que un sistema, una aplicación o un servicio deje de estar disponible para los usuarios legítimos. Lo importante aquí es entender que el atacante no busca necesariamente acceder al sistema, robar información o modificar datos, sino

romper la disponibilidad. En otras palabras, lo que se persigue es que el servicio deje de responder, responda muy lentamente o rechace conexiones que en condiciones normales sí aceptaría.

Lo que hemos aprendido en esta práctica es que un ataque DoS puede afectar a distintos niveles del sistema. No siempre se basa en “consumir mucha CPU”; a veces lo que se agota son otros recursos, como la capacidad de la red, la tabla de conexiones o la propia lógica de un servicio.

4.1 DoS VOLUMÉTRICO

La primera gran variante es el DoS volumétrico. En este tipo de ataque lo que se intenta es inundar la red o el servidor con una cantidad enorme de tráfico. Lo que hemos de imaginar aquí es una saturación del canal: se envían tantos paquetes que el ancho de banda o la capacidad de procesamiento de la interfaz de red quedan sobrepasados. El resultado es que el tráfico legítimo no puede pasar con normalidad.

Este tipo de ataque no necesita explotar un fallo del protocolo ni del software; le basta con generar una carga muy alta de tráfico. En muchos casos el cuello de botella aparece antes incluso de que el servicio pueda procesar la petición.

4.2 DoS A NIVEL DE PROTOCOLO

La segunda variante es el DoS a nivel de protocolo, y aquí es donde encaja exactamente el ataque que hemos realizado en la práctica. En este caso no se busca solo enviar muchos paquetes, sino aprovechar el funcionamiento normal de protocolos como TCP para obligar al sistema a reservar recursos de forma ineficiente.

El ejemplo clásico es el SYN Flood. Cuando un cliente legítimo quiere abrir una conexión TCP con un servidor, se realiza el llamado three-way handshake o saludo de tres pasos:

- El cliente envía un paquete SYN.
- El servidor responde con SYN-ACK.
- El cliente confirma con ACK.

Solo después de ese tercer paso la conexión queda plenamente establecida.

Lo que hemos hecho en el ataque ha sido enviar una gran cantidad de paquetes SYN al puerto 80 del servidor víctima, pero sin completar el tercer paso del handshake. De ese modo, el servidor ha ido reservando recursos y dejando conexiones pendientes de completar. Cada una de esas conexiones queda en estado half-open o medio abierta. Si se acumulan demasiadas, el sistema empieza a tener dificultades para aceptar nuevas conexiones legítimas. Esta es la esencia del SYN Flood y es precisamente lo que hemos podido observar cuando ab empezó a devolver errores de conexión durante el ataque.

4.3 DoS A NIVEL DE APLICACIÓN

La tercera variante es el DoS a nivel de aplicación. Aquí el objetivo no es tanto la pila TCP/IP como la propia aplicación. Por ejemplo, en un servidor web se podrían enviar muchas peticiones HTTP complejas, o peticiones que obliguen al backend a consultar una base de datos continuamente. En estos casos el impacto se nota sobre procesos concretos del servicio, sobre el tiempo de respuesta de la aplicación o sobre recursos como memoria y CPU a nivel de usuario.

Este tipo de ataque puede ser especialmente efectivo porque desde fuera puede parecer tráfico “normal” del servicio, aunque en realidad esté diseñado para agotarlo.

4.4 DDoS

Por último, tenemos el DDoS (Distributed Denial of Service). Conceptualmente es el mismo objetivo que en un DoS clásico, pero en vez de atacar desde un solo sistema, el

tráfico malicioso se genera desde muchas máquinas simultáneamente, a menudo equipos comprometidos que forman una botnet. Esto hace que el ataque sea mucho más difícil de filtrar y mitigar, porque ya no depende de una sola IP o de una sola fuente fácilmente bloqueable.

4.5 RELACIÓN CON NUESTRA PRÁCTICA

En nuestro caso concreto, el ataque que hemos ejecutado ha sido claramente un DoS a nivel de protocolo, concretamente un SYN Flood contra el servicio Apache de la máquina víctima. Lo que hemos buscado no ha sido consumir el servicio mediante peticiones HTTP válidas ni inundar el enlace con un tráfico genérico cualquiera, sino forzar al sistema a gestionar miles de inicios de conexión TCP que no llegan a completarse. Esa es la razón por la que los indicadores más relevantes en nuestra práctica han sido el número de conexiones, los errores de conexión legítima y la captura de los paquetes SYN, más que un pico extremo de CPU sostenido.

5 DESARROLLO TÉCNICO DE LO QUE HEMOS HECHO EN EL LABORATORIO

Aunque estos apartados no son todavía el “informe técnico final” en el sentido de conclusiones globales, sí conviene dejar claro lo que hemos hecho realmente para contextualizar el resto del documento.

5.1 PREPARACIÓN DE LA MÁQUINA VÍCTIMA

Lo primero que hemos hecho ha sido preparar la máquina víctima con Ubuntu Server. Hemos instalado Apache y hemos comprobado que el servicio respondía correctamente en local. Después hemos creado un script de monitorización para registrar continuamente varios indicadores en un archivo de log:

- uso de CPU mediante sar,
- resumen de sockets y conexiones con ss -s,

- número de conexiones hacia el puerto 80,
- latencia local HTTP mediante curl.

Como no teníamos un escritorio completo ni varias terminales cómodas abiertas en la víctima, hemos decidido ejecutar el script en segundo plano con `nohup` y guardar el PID en un archivo. Lo que hemos hecho aquí ha sido convertir una limitación del entorno en una ventaja, porque los logs generados nos han servido luego como evidencia objetiva del comportamiento del servidor.

También hemos preparado la captura de tráfico con `tcpdump`, primero en una versión larga y después, al ver que el archivo crecía demasiado, en una versión limitada para obtener un `.pcap` pequeño y útil para el informe.

5.2 PREPARACIÓN DE LA MÁQUINA ATACANTE

En la máquina atacante hemos instalado y comprobado `hping3`, `apache2-utils` y `curl`. Antes de lanzar el ataque hemos verificado que el servidor web de la víctima respondía correctamente, y también hemos realizado pruebas de referencia con `ab` para tener una idea del comportamiento normal del servicio.

Una vez preparado todo, hemos lanzado el ataque con el comando de `hping3` en modo flood, enviando paquetes TCP SYN al puerto 80 del servidor víctima. Paralelamente, desde otra terminal en la máquina atacante, hemos intentado realizar peticiones legítimas al servidor mediante `ab` para observar si seguía siendo accesible bajo carga maliciosa.

5.3 LO QUE HEMOS OBSERVADO EN LAS CAPTURAS Y RESULTADOS

En la terminal del atacante se hace hping3 en flood mode enviando SYN al puerto 80.

En otra terminal, al ejecutar ab, aparece el error Connection reset by peer (104), lo que demuestra que el servidor no estaba aceptando correctamente nuevas conexiones legítimas.

En el log de aparece un valor de HTTP-CONN-COUNT de 543, lo que evidencia una acumulación muy alta de conexiones relacionadas con el puerto 80.

En ese mismo log aparece un uso de CPU en modo system cercano al 50 %, lo que tiene sentido porque quien está gestionando la presión del ataque es principalmente el kernel y la pila TCP, no una aplicación en espacio de usuario.

Estos resultados encajan muy bien con un ataque SYN Flood real en entorno de laboratorio: muchas conexiones pendientes, rechazo de tráfico legítimo y actividad significativa del kernel gestionando paquetes TCP.

6 CONTRAMEDIDAS BÁSICAS

Después de comprobar cómo afecta un SYN Flood al servidor, lo lógico es preguntarnos qué medidas básicas podemos aplicar para reducir el impacto. Aquí no se trata de encontrar una defensa perfecta, porque no existe una solución única que elimine por completo este tipo de riesgo, sino de entender qué mecanismos ayudan a que el sistema sea más resistente.

6.1 SYN COOKIES

Una de las contramedidas más importantes frente a un SYN Flood son las SYN cookies. Lo que hacen es evitar que el sistema reserve inmediatamente recursos completos para cada solicitud SYN entrante. En vez de almacenar el estado de la conexión desde el primer momento, el sistema codifica cierta información en la respuesta SYN-ACK y solo crea realmente la conexión cuando el cliente completa correctamente el handshake.

Lo importante aquí es que esta técnica reduce el impacto de las conexiones half-open, porque impide que un gran volumen de SYN incompletos llene rápidamente la cola de conexiones pendientes.

6.2 LIMITACIÓN DE CONEXIONES Y RATE LIMITING

Otra medida básica consiste en limitar el número de conexiones nuevas o la tasa de peticiones por dirección IP. Esto puede hacerse con reglas de firewall. Lo que hemos de entender aquí es que, si una misma fuente intenta abrir una cantidad anormal de conexiones en muy poco tiempo, ese comportamiento puede bloquearse o ralentizarse antes de que llegue al servicio.

Esta medida no siempre es suficiente por sí sola, especialmente en ataques distribuidos, pero en un laboratorio como el nuestro y en entornos pequeños puede marcar una diferencia muy importante.

6.3 FIREWALLS E IDS/IPS

Los firewalls y los sistemas de detección o prevención de intrusiones (IDS/IPS) permiten identificar patrones de tráfico anómalos. Por ejemplo, un volumen inusual de paquetes SYN hacia un mismo puerto, o un número excesivo de conexiones que no llegan a completarse, son señales muy claras de un posible ataque de este tipo.

Lo que hemos aprendido con la práctica es que no basta con tener el servicio levantado; también es fundamental contar con mecanismos que vigilen cómo se comporta la red alrededor de ese servicio.

6.4 AJUSTE DE PARÁMETROS DEL KERNEL TCP

Otra línea de defensa básica consiste en ajustar ciertos parámetros del sistema operativo relacionados con TCP, como el tamaño de la cola de conexiones pendientes o el tiempo que se mantienen conexiones incompletas. Esto no elimina el ataque, pero sí puede dar al servidor mayor capacidad de absorción frente a un pico de conexiones medio abiertas.

6.5 BALANCEADORES Y PROTECCIÓN PERIMETRAL

En entornos reales, donde el servicio tiene más exposición, suele ser habitual emplear balanceadores de carga, proxies inversos o servicios especializados de mitigación DDoS. Lo que hacen estos elementos es absorber, distribuir o filtrar el tráfico antes de que llegue al servidor final.

6.6 MONITORIZACIÓN CONTINUA

Una de las lecciones más claras de esta práctica es que la monitorización es una contramedida básica en sí misma. Herramientas como htop, sar, ss o tcpdump no bloquean el ataque, pero sí nos permiten detectar rápidamente lo que está ocurriendo. Gracias a ellas hemos podido comprobar que el servidor estaba acumulando conexiones, que el kernel estaba procesando mucho tráfico y que el servicio legítimo empezaba a fallar.

7 PARTE PRÁCTICA

Obtenemos la dirección IP de ambas máquinas y comprobamos que tengamos conexión entre estas.

```
glox_server@glox-server:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast
    link/ether 00:0c:29:1d:a5:7b brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.18.134/24 metric 100 brd 192.168.18.255 scope global ens33
        valid_lft 1404sec preferred_lft 1404sec
    inet6 fe80::20c:29ff:fe1d:a57b/64 scope link
        valid_lft forever preferred_lft forever
glox_server@glox-server:~$
```

```
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast
up default qlen 1000
    link/ether 00:0c:29:ea:bf:da brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.18.130/24 brd 192.168.18.255 scope global ens33
        valid_lft 1428sec preferred_lft 1428sec
    inet6 fe80::fc74:a8e5:c11c:1a51/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

```
[glox@parrot]~$
```

```
[glox@parrot]~$ ping -c 1 192.168.18.134
PING 192.168.18.134 (192.168.18.134) 56(84) bytes of data:
64 bytes from 192.168.18.134: icmp_seq=1 ttl=64 time=0.423 ms

--- 192.168.18.134 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.423/0.423/0.423/0.000 ms
[glox@parrot]~$
```

7.1 APACHE

Ahora que ya sabemos que tenemos conexión, instalamos apache y comprobamos que está encendido.

```
glox_server@glox-server:~$ systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-03-13 15:59:48 UTC; 7min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 1256 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
 Main PID: 1283 (apache2)
    Tasks: 55 (limit: 4431)
   Memory: 8.1M (peak: 8.8M)
      CPU: 79ms
   CGroup: /system.slice/apache2.service
           └─1283 /usr/sbin/apache2 -k start
             └─1285 /usr/sbin/apache2 -k start
               └─1286 /usr/sbin/apache2 -k start

mar 13 15:59:48 glox-server systemd[1]: Starting apache2.service - The Apache HTTP Server...
mar 13 15:59:48 glox-server apachectl[1275]: AH00558: apache2: Could not reliably determine the s
mar 13 15:59:48 glox-server systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-17/17 (END)
```

Comprobamos que nuestro parrot puede llegar a la web.

```
[glox@parrot]~$ curl http://192.168.18.134
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/1
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
      * {
        margin: 0px 0px 0px 0px;
```

Vamos a ver ahora las estadísticas normales que hay en el servidor sin un ataque DoS

7.2 STATS NORMALES

Miramos el htop para ver el uso de recursos de hardware, vemos que es mínimo, ya que no hay muchos procesos corriendo.

```
0% Tasks: 27, 77 thr, 201 kthr: 1 running
1% Load average: 0.02 0.01 0.00
2% Uptime: 00:30:57
3%
Mem[|||||] 259M/3.68G
Swap[ ] 0K/3.68G

Main T/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
1 root 20 0 22036 13284 9580 S 0.0 0.3 0:02.06 /sbin/init
466 root 19 -1 50516 15804 14620 S 0.0 0.4 0:00.22 /usr/lib/systemd/systemd-journald
502 root RT 0 346M 27560 8760 S 0.0 0.7 0:00.12 /sbin/multipathd -d -s
531 root 20 0 31320 10060 5056 S 0.0 0.3 0:00.20 /usr/lib/systemd/systemd-udev
534 root 20 0 346M 27560 8760 S 0.0 0.7 0:00.00 /sbin/multipathd -d -s
535 root RT 0 346M 27560 8760 S 0.0 0.7 0:00.00 /sbin/multipathd -d -s
536 root RT 0 346M 27560 8760 S 0.0 0.7 0:00.00 /sbin/multipathd -d -s
537 root RT 0 346M 27560 8760 S 0.0 0.7 0:00.00 /sbin/multipathd -d -s
538 root RT 0 346M 27560 8760 S 0.0 0.7 0:00.12 /sbin/multipathd -d -s
539 root RT 0 346M 27560 8760 S 0.0 0.7 0:00.00 /sbin/multipathd -d -s
705 systemd-ne 20 0 19000 9492 6324 S 0.0 0.2 0:00.05 /usr/lib/systemd/systemd-networkd
717 systemd-re 20 0 21584 13112 10816 S 0.0 0.3 0:00.13 /usr/lib/systemd/systemd-resolved
727 systemd-ti 20 0 91024 7924 6948 S 0.0 0.2 0:00.05 /usr/lib/systemd/systemd-timesyncd
770 systemd-ti 20 0 91024 7924 6948 S 0.0 0.2 0:00.00 /usr/lib/systemd/systemd-timesyncd
782 root 20 0 53468 12652 10492 S 0.0 0.3 0:00.05 /usr/bin/vgaauthservice
783 root 20 0 236M 9524 8036 S 0.0 0.2 0:01.93 /usr/bin/vmtotlsd
843 root 20 0 236M 9524 8036 S 0.0 0.2 0:00.00 /usr/bin/vmtotlsd
847 messagebus 20 0 9796 5604 4688 S 0.0 0.1 0:00.05 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-o
852 polkitd 20 0 300M 8020 7124 S 0.0 0.2 0:00.05 /usr/lib/polkit-1/polkitd --no-debug
861 root 20 0 18144 8928 7816 S 0.0 0.2 0:00.06 /usr/lib/systemd/systemd-logind
862 root 20 0 236M 9524 8036 S 0.0 0.2 0:00.08 /usr/bin/vmtotlsd
863 root 20 0 236M 9524 8036 S 0.0 0.2 0:00.00 /usr/bin/vmtotlsd
878 root 20 0 457M 13648 11432 S 0.0 0.4 0:00.10 /usr/libexec/udisks2/udisksd
891 syslog 20 0 217M 6456 4592 S 0.0 0.2 0:00.04 /usr/sbin/rsyslogd -n -iNONE
899 root 20 0 107M 23284 13748 S 0.0 0.6 0:00.13 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
912 root 20 0 457M 13648 11432 S 0.0 0.4 0:00.02 /usr/libexec/udisks2/udisksd
919 root 20 0 457M 13648 11432 S 0.0 0.4 0:00.00 /usr/libexec/udisks2/udisksd
924 root 20 0 457M 13648 11432 S 0.0 0.4 0:00.00 /usr/libexec/udisks2/udisksd
929 syslog 20 0 217M 6456 4592 S 0.0 0.2 0:00.00 /usr/sbin/rsyslogd -n -iNONE
930 syslog 20 0 217M 6456 4592 S 0.0 0.2 0:00.00 /usr/sbin/rsyslogd -n -iNONE
931 syslog 20 0 217M 6456 4592 S 0.0 0.2 0:00.00 /usr/sbin/rsyslogd -n -iNONE
936 polkitd 20 0 300M 8020 7124 S 0.0 0.2 0:00.02 /usr/lib/polkit-1/polkitd --no-debug
937 polkitd 20 0 300M 8020 7124 S 0.0 0.2 0:00.00 /usr/lib/polkit-1/polkitd --no-debug
939 polkitd 20 0 300M 8020 7124 S 0.0 0.2 0:00.00 /usr/lib/polkit-1/polkitd --no-debug
987 root 20 0 382M 12852 10780 S 0.0 0.3 0:00.08 /usr/sbin/ModemManager
1021 root 20 0 457M 13648 11432 S 0.0 0.4 0:00.00 /usr/libexec/udisks2/udisksd
1046 root 20 0 382M 12852 10780 S 0.0 0.3 0:00.00 /usr/sbin/ModemManager
1056 root 20 0 382M 12852 10780 S 0.0 0.3 0:00.00 /usr/sbin/ModemManager
1060 root 20 0 457M 13648 11432 S 0.0 0.4 0:00.00 /usr/libexec/udisks2/udisksd
~!Help ~F2Setup ~F3Search ~F4Listen ~F5Tree ~F6SortBy ~F7Nice ~F8Nice ~F9Kill ~F10Quit
```

Aquí vemos que no hay apenas solicitudes TCP, sólo hay 6.

```
Every 1.0s: ss -s

Total: 152
TCP: 6 (estab 0, closed 1, orphaned 0, timewait 0)

Transport Total IP IPv6
RAW 1 0 1
UDP 4 4 0
TCP 5 3 2
INET 10 7 3
FRAG 0 0 0
```

Como tenemos un Ubuntu server, vamos a realizar un script con todos los comandos que se ejecute en segundo plano para tenerlo todo en un mismo archivo y poder ver de forma dinámica los cambios y las nuevas peticiones.

```
GNU nano 7.2 monitor_victima.sh
#!/bin/bash

LOGDIR="$HOME/act4-victima/logs"
mkdir -p $LOGDIR

while true; do
    echo "==== $(date '+%F %T') ====" >> "$LOGDIR/monitor.log"

    echo "[CPU]" >> "$LOGDIR/monitor.log"
    sar -u 1 1 >> "$LOGDIR/monitor.log"

    echo "[SS-SUMMARY]" >> "$LOGDIR/monitor.log"
    ss -s >> "$LOGDIR/monitor.log"

    echo "[HTTP-COMN-COUNT]" >> "$LOGDIR/monitor.log"
    ss -ant | grep ':80' | wc -l >> "$LOGDIR/monitor.log"

    echo "[LOCAL-HTTP-LATENCY]" >> "$LOGDIR/monitor.log"
    curl -o /dev/null -s -w "%{time_total}\n" http://localhost >> "$LOGDIR/monitor.log"

    echo "" >> "$LOGDIR/monitor.log"
    sleep 1
done
```

7.3 AB

Usamos la tool ab para comprobar la capacidad de peticiones inicial que tiene nuestro Ubuntu server.

```
[glox@parrot]~/act4-atacante
$ ab -n 1000 -c 50 http://192.168.18.134/ > baseline_ab.txt
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Completed 600 requests
Completed 700 requests
Completed 800 requests
Completed 900 requests
Completed 1000 requests
Finished 1000 requests
[glox@parrot]~/act4-atacante
$
```

7.4 DoS

Vamos a realizar el DoS:

```
[x]-[glox@parrot]-[~/act4-atacante]
$ sudo hping3 -S --flood -V -p 80 192.168.18.134
using ens33, addr: 192.168.18.130, MTU: 1500
HPING 192.168.18.134 (ens33 192.168.18.134): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

Comprobamos la capacidad de peticiones durante el ataque:

```
[x]-[glox@parrot]-[~/act4-atacante]
$ ab -n 500 -c 50 http://192.168.18.134:80/
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 192.168.18.134 (be patient)
apr_socket_recv: Connection reset by peer (104)
Total of 26 requests completed
```

Como vemos, antes teníamos 7. Después del ataque tenemos muchas más.

```
==== 2026-03-13 16:45:44 ====
[CPU]
Linux 6.8.0-101-generic (glox-server) 13/03/26

16:45:44      CPU      %user      %nice      %system      %i
16:45:45      all       0,00       0,00       0,00
Average:      all       0,00       0,00       0,00
[SS-SUMMARY]
Total: 152
TCP:  7 (estab 0, closed 2, orphaned 0, timewait 1)

Transport Total      IP      IPv6
RAW        1        0        1
UDP        4        4        0
TCP        5        3        2
INET      10        7        3
FRAG       0        0        0
```



```

==== 2026-03-13 16:45:46 ====
[CPU]
Linux 6.8.0-101-generic (glox-server)    13/03/26    _x86_

16:45:46      CPU      %user      %nice      %system      %iowait
16:45:47      all      0,00      0,00      0,25      0,25
Average:      all      0,00      0,00      0,25      0,25
[SS-SUMMARY]
Total: 152
TCP:   8 (estab 0, closed 3, orphaned 0, timewait 2)

```

```

17:39:27      CPU      %user
17:39:28      all      0,00
Average:      all      0,00
[SS-SUMMARY]
Total: 162
TCP:   37 (estab 0, closed 32,

```

```

17:39:29      CPU      %user      %nice      %s
17:39:30      all      0,00      0,00      5
Average:      all      0,00      0,00      5
[SS-SUMMARY]
Total: 162
TCP:   35 (estab 0, closed 30, orphaned 0, t

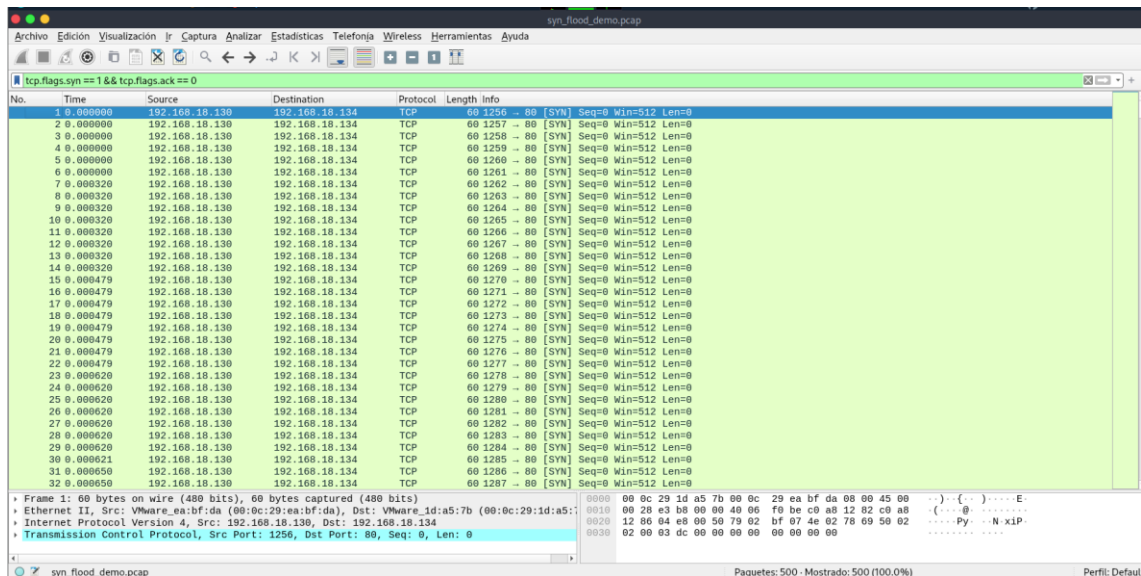
```

En estas capturas de pantalla se aprecia la diferencia entre no recibir peticiones TCP y, luego cuando ya se realiza el DoS que la cantidad de peticiones y conexiones TCP aumenta exponencialmente.

Es tal la cantidad de peticiones, que llega un punto en el que el servidor deja de responder a los pings y a las peticiones, ya que el servidor estaba totalmente fuera de servicio.

7.5 WIRESHARK

Hemos capturado también las peticiones con tcpdump y, las luego las hemos analizado con wireshark. Si aplicamos el filtro, veremos que todas las peticiones son SYN, que es un tipo de DoS que hemos comentado previamente.



Aquí wireshark nos muestra que el pcap tiene 500 tramas, por lo que hemos realizado más de 500 peticiones TCP al servidor web apache.

Estadísticas

Medida	Capturado
Paquetes	500
Espacio de tiempo, s	0.038

8 INCIDENCIAS TÉCNICAS Y SOLUCIONES

Uno de los apartados más valiosos del trabajo es explicar no solo lo que ha salido bien, sino también los problemas reales que nos hemos encontrado y cómo los hemos resuelto. Esto hace que el informe sea mucho más creíble y técnico.

8.1 LIMITACIÓN DEL ENTORNO UBUNTU SERVER

La primera incidencia importante ha sido el propio entorno de la víctima. Al trabajar con Ubuntu Server, no contábamos con un entorno gráfico cómodo para tener muchas terminales abiertas a la vez y observar varias herramientas en paralelo.

8.1.1 ¿Qué hemos hecho?

En vez de intentar forzar una forma de trabajo incómoda, lo que hemos hecho ha sido redirigir la salida de los comandos a archivos y ejecutar los procesos en segundo plano. Así hemos podido mantener funcionando al mismo tiempo el script de monitorización y la captura de red, sin necesidad de tener múltiples sesiones interactivas abiertas.

8.1.2 Solución aplicada

Hemos creado un script que registra CPU, conexiones y latencia en un log, y lo hemos lanzado con nohup. Esto ha sido una solución técnica correcta y además muy útil para documentar el comportamiento del sistema.

8.2 TAMAÑO EXCESIVO DEL ARCHIVO .PCAP

Otra incidencia importante ha aparecido al capturar tráfico de red durante el ataque. Como el SYN Flood genera una cantidad enorme de paquetes, el archivo .pcap inicial terminó siendo demasiado grande y tardaba mucho en abrirse con Wireshark.

8.2.1 ¿Qué hemos observado?

Lo que hemos visto aquí es que una captura completa del ataque no siempre es práctica para el análisis. Aunque técnicamente contiene toda la información, en realidad dificulta el trabajo porque pesa demasiado y hace más lento el filtrado.

8.2.2 ¿Qué hemos hecho para resolverlo?

Hemos decidido realizar una segunda captura mucho más pequeña, filtrando solo los paquetes SYN hacia el puerto 80 y limitando además el número de paquetes con la opción -c de tcpdump.

Con esa solución hemos obtenido un .pcap mucho más manejable, suficiente para mostrar en el informe la evidencia esencial del ataque: una secuencia de paquetes TCP SYN dirigidos desde la máquina atacante hacia la víctima. Esta incidencia y su solución encajan exactamente con lo que ya has recogido en el documento base.

9 CONCLUSIÓN

En esta práctica hemos comprobado de forma experimental cómo un ataque DoS a nivel de protocolo puede afectar de manera directa a la disponibilidad de un servidor web. Lo que hemos hecho no ha sido simplemente ejecutar un comando y observar un resultado aislado, sino montar un pequeño entorno técnico donde hemos podido registrar y analizar el impacto del ataque desde varios puntos de vista.

Hemos visto que un ataque DoS no siempre se traduce únicamente en un uso extremo de CPU o memoria. En el caso concreto del SYN Flood, el efecto principal aparece en la gestión de conexiones TCP incompletas, que acaban dificultando o impidiendo la atención de nuevas conexiones legítimas. También hemos comprobado que una parte importante del trabajo técnico está en saber monitorizar, interpretar y documentar lo que ocurre.

Además, las incidencias técnicas que hemos ido resolviendo durante la práctica han sido útiles para comprender mejor las herramientas y adaptar el procedimiento al entorno real que teníamos disponible. El uso de scripts en segundo plano, la reducción del tamaño del

.pcap y la interpretación correcta de los errores de ApacheBench forman parte del aprendizaje real del laboratorio.

En definitiva, lo que hemos demostrado aquí es que incluso en un entorno pequeño y controlado es posible reproducir con claridad el comportamiento básico de un ataque.